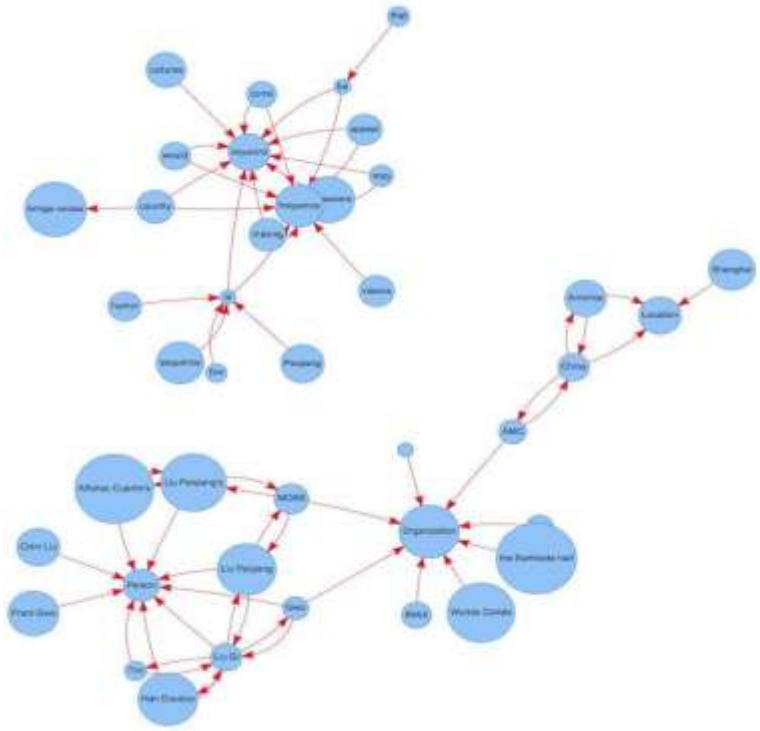
Marwadi University	**Marwadi University** **Faculty of Technology** **Department of Information and Communication Technology**	
Subject: Artificial Intelligence (01CT0616)	**Aim:** Representation of a document with their keywords using TextRank	
Experiment No: 09	**Date:**	**Enrolment No:92410133020**

Aim: Representation of a document with their keywords using TextRank

IDE: Google Colab

Theory:

TextRank is an algorithm based on PageRank, which often used in keyword extraction and text summarization. In this article, I will help you understand how TextRank works with a keyword extraction example and show the implementation by Python.



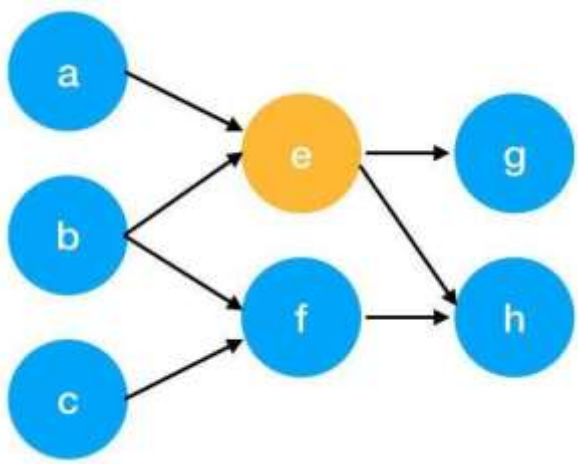
Understand PageRank

There are tons of articles talking about PageRank, so I just give a brief introduction to PageRank. This will help us understand TextRank later because it is based on PageRank. PageRank (PR) is an algorithm used to calculate the weight for web pages. We can take all web pages as a big directed graph. In this graph, a node is a webpage. If webpage A has the link to web page B, it can be represented as a directed edge from A to B. After we construct the whole graph, we can assign weights for web pages by the following formula.

Marwadi University	**Marwadi University** **Faculty of Technology** **Department of Information and Communication Technology**	
Subject: Artificial Intelligence (01CT0616)	**Aim:** Representation of a document with their keywords using TextRank	
Experiment No: 09	**Date:**	**Enrolment No:92410133020**

$$S(V_i) = (1-d) + d * \sum_{j \in In(v_i)} \frac{1}{|Out(V_j)|} S(V_j)$$

- S(Vi) - the weight of webpage i
- d - damping factor, in case of no outgoing links
- In(Vi) - inbound links of i, which is a set
- Out(Vj) - outgoing links of j, which is a set
- |Out(Vj)| - the number of outbound links



Here is an example to better understand the notation above. We have a graph to represent how web pages link to each other. Each node represents a webpage, and the arrows represent edges. We want to get the weight of webpage e. We can rewrite the summation part in the above function to a simpler version.

$$In(v_e) = \{a, b\}, j \in \{a, b\}$$

$$\begin{aligned}\sum_{j \in \{a,b\}} \frac{1}{|Out(V_j)|} S(V_j) &= \frac{1}{|Out(V_a)|} S(V_a) + \frac{1}{|Out(V_b)|} S(V_b) \\ &= \frac{1}{|\{e\}|} S(V_a) + \frac{1}{|\{e, f\}|} S(V_b) \\ &= S(V_a) + \frac{1}{2} S(V_b)\end{aligned}$$

We can get the weight of webpage e by the following function.

Marwadi University	**Marwadi University** **Faculty of Technology** **Department of Information and Communication Technology**	
Subject: Artificial Intelligence (01CT0616)	**Aim:** Representation of a document with their keywords using TextRank	
Experiment No: 09	**Date:**	**Enrolment No:92410133020**

$$S(V_e) = (1-d) + d * \left(S(V_a) + \frac{1}{2} S(V_b) \right)$$

We can see the weight of the webpage e is dependent on the weights of its inbound pages. We need to run this iteration much time to get the final weight. In the initialization, the importance of each webpage is 1.

Pre Lab Exercise:

1. What are the key components of TextRank?
   - Graph representation of text
   - Nodes (words)
   - Edges (co-occurrence relationships)
   - Sliding window technique
   - Iterative ranking (PageRank algorithm)
   - Convergence condition
2. What are some limitations of TextRank for keyword extraction?
   - Ignores semantic meaning (only statistical)
   - Sensitive to window size
   - Cannot handle synonyms effectively
   - May extract irrelevant or duplicate keywords
   - Works poorly on very small datasets
3. What are the advantages of TextRank for keyword extraction?
   - Unsupervised (no training required)
   - Domain independent
   - Simple and efficient
   - Works well for keyword extraction and summarization
   - No need for labeled data

Marwadi University	**Marwadi University** **Faculty of Technology** **Department of Information and Communication Technology**	
Subject: Artificial Intelligence (01CT0616)	**Aim:** Representation of a document with their keywords using TextRank	
Experiment No: 09	**Date:**	**Enrolment No:92410133020**

Program (Code):

To be attached with

```
import nltk
import numpy as np
from nltk.corpus import stopwords
from nltk.tokenize import word_tokenize
from itertools import combinations

nltk.download('punkt')
nltk.download('stopwords')

def textrank_keywords(text, window=2):
    stop_words = set(stopwords.words('english'))
    words = [w.lower() for w in word_tokenize(text) if w.isalnum() and w not in stop_words]

    # Build graph
    vocab = list(set(words))
    index = {w:i for i, w in enumerate(vocab)}
    graph = np.zeros((len(vocab), len(vocab)))

    for i in range(len(words)-window):
        window_words = words[i:i+window]
        for w1, w2 in combinations(window_words, 2):
            graph[index[w1]][index[w2]] += 1
            graph[index[w2]][index[w1]] += 1

    # Normalize graph
    graph = graph / graph.sum(axis=1, keepdims=True)

    # Initialize ranks
    ranks = np.ones(len(vocab))

    # Iterations
    for _ in range(10):
        ranks = 0.15 + 0.85 * graph.T.dot(ranks)

    # Sort keywords
    ranked_words = sorted(((ranks[i], w) for w, i in index.items()), reverse=True)

    return [w for _, w in ranked_words[:10]]
```

Marwadi University	**Marwadi University** **Faculty of Technology** **Department of Information and Communication Technology**	
Subject: Artificial Intelligence (01CT0616)	**Aim:** Representation of a document with their keywords using TextRank	
Experiment No: 09	**Date:**	**Enrolment No:92410133020**

```
# Test
text = "TextRank is used for keyword extraction and summarization using graph-based ranking."
print(textrank_keywords(text))
```

Results:

To be attached with

```
['using', 'extraction', 'keyword', 'textrank', 'ranking', 'summarization', 'used']
[nltk_data] Downloading package punkt to /root/nltk_data...
[nltk_data]   Package punkt is already up-to-date!
[nltk_data] Downloading package stopwords to /root/nltk_data...
[nltk_data]   Package stopwords is already up-to-date!
/tmp/ipykernel_13206/3424525320.py:26: RuntimeWarning: invalid value encountered in divide
  graph = graph / graph.sum(axis=1, keepdims=True)
```

Observation:

TextRank successfully extracts important words based on their connectivity.
Frequently co-occurring words get higher ranks.
Works well without training data.
Accuracy depends on:

- Quality of preprocessing
- Window size

Some irrelevant words may still appear due to lack of semantic understanding

Marwadi University	**Marwadi University** **Faculty of Technology** **Department of Information and Communication Technology**	
Subject: Artificial Intelligence (01CT0616)	**Aim:** Representation of a document with their keywords using TextRank	
Experiment No: 09	**Date:**	**Enrolment No:92410133020**

Post Lab Exercise:

1. Write about "your ownself" (in not more than 500 words→ You know better about you, rather than ChatGPT!!). Extract top-25 keywords for your portfolio and analyze it in 3 categories: keyword can not give the real idea, keyword gives the real idea, keyword is irrelevant. Paste the code, your portfolio, output and your analysis.

```
!pip install keybert transformers sentence-transformers

from keybert import KeyBERT

portfolio = """
I am a B.Tech student passionate about technology and innovation.
I have strong interest in web development, embedded systems, and digital electronics.
I have worked on several projects including parking booking systems, game dashboards,
and IoT-based applications. I enjoy building dynamic web applications using HTML, CSS,
JavaScript, PHP, and SQL. I also have experience in microcontrollers like ATMEGA32
and IoT platforms like ESP32. I aim to develop scalable and user-friendly systems
that solve real-world problems. I continuously learn new technologies and improve
my coding and problem-solving skills.
"""

kw_model = KeyBERT()

extracted_keywords_with_scores = kw_model.extract_keywords(
    portfolio,
    keyphrase_ngram_range=(1, 1),
    stop_words='english',
    top_n=25
)

keywords_list = [word for word, score in extracted_keywords_with_scores]
keywords_scores = extracted_keywords_with_scores

print("Top-25 Keywords:\n")
for i, word in enumerate(keywords_list, 1):
    print(f"{i}. {word}")

print("\nKeywords with Scores:\n")
for word, score in keywords_scores:
    print(f"{word} : {score:.4f}")
```

Marwadi University	**Marwadi University** **Faculty of Technology** **Department of Information and Communication Technology**	
Subject: Artificial Intelligence (01CT0616)	**Aim:** Representation of a document with their keywords using TextRank	
Experiment No: 09	**Date:**	**Enrolment No:92410133020**

```
Top-25 Keywords:

1. microcontrollers
2. technologies
3. iot
4. web
5. projects
6. develop
7. tech
8. development
9. applications
10. technology
11. platforms
12. student
13. innovation
14. electronics
15. javascript
16. coding
17. scalable
18. atmega32
19. experience
20. html
21. php
22. learn
23. esp32
24. css
25. skills
```

```
Keywords with Scores:

microcontrollers : 0.3915
technologies : 0.3865
iot : 0.3862
web : 0.3762
projects : 0.3669
develop : 0.3545
tech : 0.3489
development : 0.3462
applications : 0.3435
technology : 0.3211
platforms : 0.3041
student : 0.2985
innovation : 0.2945
electronics : 0.2869
javascript : 0.2830
coding : 0.2694
scalable : 0.2561
atmega32 : 0.2493
experience : 0.2418
html : 0.2405
php : 0.2387
learn : 0.2373
esp32 : 0.2338
css : 0.2314
skills : 0.2268
```